

SIMATS ENGINEERING
CO1 -ASSESSMENT TOOL 2

NAME: B. Harsha Vardhan

Reg.no: 192425201

1. A logistics company is developing an RL-based warehouse robot that learns to pick, pack, and transport items efficiently while avoiding obstacles and minimizing delays. Analyze how the components of a Markov Decision Process (states, actions, rewards, policy, and environment) can be defined for this system. Design a reward function that balances speed, accuracy, and safety, and evaluate how reward design influences the robot's learning and behavior. Learning program.

CODE :

```
import random

states = [
    "Start",
    "Move",
    "Pick",
    "Pack",
    "Deliver",
    "Goal"
]

actions = [
    "Move",
    "Pick",
    "Pack",
    "Deliver"
]

current_state = "Start"
reward = 0
steps = 0

print("Warehouse Robot Reinforcement Learning")

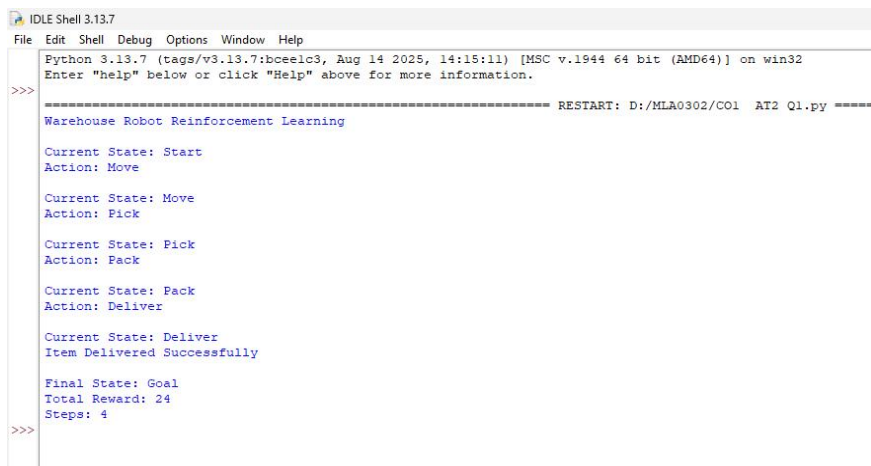
while current_state != "Goal":
    print("\nCurrent State:", current_state)
    if current_state == "Start": action = "Move"
```

```

elif current_state == "Move":
    action = random.choice(["Move", "Pick"])
elif current_state == "Pick":
    action = "Pack" elif current_state == "Pack":
    action = "Deliver"
elif current_state == "Deliver":
    current_state = "Goal"
    reward += 10
    print("Item Delivered Successfully")
    break
print("Action:", action) if action == "Move":
obstacle = random.randint(0, 4)
if obstacle == 0:
    print("Obstacle Found")
    reward -= 5
else:
    reward += 1
    current_state = "Move" elif action == "Pick":
reward += 5
    current_state = "Pick"
elif action == "Pack":
    reward += 3
    current_state = "Pack"
elif action == "Deliver":
    reward += 5
    current_state = "Deliver" steps += 1 if steps > 20:
    break
print("\nFinal State:", current_state)
print("Total Reward:", reward)
print("Steps:", steps)

```

OUTPUT :



```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:boe1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: D:/MLA0302/CO1 AT2 Q1.py =====
Warehouse Robot Reinforcement Learning
Current State: Start
Action: Move
Current State: Move
Action: Pick
Current State: Pick
Action: Pack
Current State: Pack
Action: Deliver
Current State: Deliver
Item Delivered Successfully
Final State: Goal
Total Reward: 24
Steps: 4
>>>
```

2. A streaming platform wants to use Reinforcement Learning to recommend movies dynamically based on user preferences and viewing history. Apply RL concepts to construct a suitable state representation, action space, and reward function. Analyze how different reward signals affect user engagement and evaluate the trade-off between exploration and exploitation.

CODE :

```
import random

movies = ["Action", "Comedy", "Drama", "Horror", "Sci-Fi"]
q_values = [0, 0, 0, 0, 0]
epsilon = 0.2
episodes = 20
total_reward = 0

print("Movie Recommendation using Reinforcement Learning\n")

for episode in range(episodes):
    if random.random() < epsilon:
        action = random.randint(0, len(movies) - 1)
        method = "Exploration"
    else:
        action = q_values.index(max(q_values))
        method = "Exploitation"

    reward = random.choice([1, 2, 5, -1])
    q_values[action] = q_values[action] + 0.1 * (reward - q_values[action])
    total_reward += reward

    print("Episode:", episode + 1)
    print("Method:", method)
    print("Recommended Movie:", movies[action])
```

```

print("Reward:", reward)

print("Q Values:", [round(x, 2) for x in q_values])

print()print("Training Completed")

print("Total Reward:", total_reward)

best = q_values.index(max(q_values))

print("Best Recommended Category:", movies[best])

```

OUTPUT :



```

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help

Episode: 15
Method: Exploitation
Recommended Movie: Action
Reward: 1
Q Values: [0.66, 0.1, 0.41, 0.5, 0.2]

Episode: 16
Method: Exploitation
Recommended Movie: Action
Reward: 5
Q Values: [1.1, 0.1, 0.41, 0.5, 0.2]

Episode: 17
Method: Exploitation
Recommended Movie: Action
Reward: -1
Q Values: [0.89, 0.1, 0.41, 0.5, 0.2]

Episode: 18
Method: Exploitation
Recommended Movie: Action
Reward: -1
Q Values: [0.7, 0.1, 0.41, 0.5, 0.2]

Episode: 19
Method: Exploitation
Recommended Movie: Action
Reward: -1
Q Values: [0.53, 0.1, 0.41, 0.5, 0.2]

Episode: 20
Method: Exploitation
Recommended Movie: Action
Reward: 5
Q Values: [0.98, 0.1, 0.41, 0.5, 0.2]

Training Completed
Total Reward: 33
Best Recommended Category: Action
>>>

```

3. An autonomous agricultural system uses RL to optimize irrigation and fertilizer usage based on soil conditions and weather forecasts. Design an RL framework by defining states, actions, and rewards. Analyze the effect of delayed rewards on crop yield optimization and justify the choice of a suitable learning algorithm.

CODE :

```

import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)

state, info = env.reset()

print("Initial State:", state)

total_reward = 0

for step in range(7):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

```

```

print("\nStep:", step + 1)
print("Current State:", state)
print("Action Taken:", action)
print("Next State:", next_state)
print("Reward:", reward)
total_reward += reward

state = next_state

if terminated or truncated:

    print("\nEpisode Finished")

    break

env.close()

print("\nTotal Reward:", total_reward)

```

OUTPUT :



```

DLE Shell 3.13.7
Edit Shell Debug Options Window Help
Action: Decrease Water
Reward: -5

Episode: 4
State: Wet Soil
Action: Add Fertilizer
Reward: -5

Episode: 5
State: Dry Soil
Action: Decrease Water
Reward: -5

Episode: 6
State: Wet Soil
Action: Increase Water
Reward: -5

Episode: 7
State: Wet Soil
Action: Add Fertilizer
Reward: -5

Episode: 8
State: Normal Soil
Action: Add Fertilizer
Reward: 8

Episode: 9
State: Wet Soil
Action: No Action
Reward: -2

Episode: 10
State: Dry Soil
Action: Decrease Water
Reward: -5

Training Completed
Total Reward: -34

```

4. A cybersecurity system uses RL to detect and respond to network attacks in real time. Analyze how states and environment can be modeled for intrusion detection. Design appropriate actions and reward functions, and evaluate the challenges of sparse rewards and real-time decision-making.

CODE :

```

import random

states = [

    "Normal Traffic",

```

```

    "Suspicious Traffic",
    "Attack Detected"
]actions = [
    "Allow",
    "Monitor",
    "Block"
]
reward = 0
episodes = 10
print("Cybersecurity Intrusion Detection using Reinforcement Learning\n")
for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
    print("Episode:", episode + 1)
    print("State:", state)
    print("Action:", action)
    if state == "Attack Detected" and action == "Block":
        r = 10
    elif state == "Suspicious Traffic" and action == "Monitor":
        r = 6
    elif state == "Normal Traffic" and action == "Allow":
        r = 5
    else:
        r = -5
    reward += r
    print("Reward:", r)
    print()
print("Training Completed")
print("Total Reward:", reward)

```

OUTPUT :

```
IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Action: Allow
Reward: 5

Episode: 4
State: Normal Traffic
Action: Monitor
Reward: -5

Episode: 5
State: Attack Detected
Action: Block
Reward: 10

Episode: 6
State: Suspicious Traffic
Action: Block
Reward: -5

Episode: 7
State: Attack Detected
Action: Allow
Reward: -5

Episode: 8
State: Attack Detected
Action: Block
Reward: 10

Episode: 9
State: Normal Traffic
Action: Monitor
Reward: -5

Episode: 10
State: Normal Traffic
Action: Block
Reward: -5

Training Completed
Total Reward: 0
>>>
```

5. A ride-sharing company plans to use RL to optimize driver allocation and dynamic pricing strategies. Apply RL concepts to define states, actions, and rewards. Analyze how environmental uncertainty affects learning and evaluate ethical concerns related to pricing strategies.

CODE:

```
import random

states = [
    "Low Demand",
    "Medium Demand",
    "High Demand"
]

actions = [
    "Assign Driver",
    "Increase Price",
    "Keep Price"
]

total_reward = 0
episodes = 10

print("Ride Sharing using Reinforcement Learning\n")

for episode in range(episodes):
    state = random.choice(states)
    action = random.choice(actions)
```

```

print("Episode:", episode + 1)
print("State:", state)
print("Action:", action)
if state == "High Demand" and action == "Increase Price":
    reward = 10
elif state == "Medium Demand" and action == "Assign Driver":
    reward = 8
elif state == "Low Demand" and action == "Keep Price":
    reward = 6
else:
    reward = -5
total_reward += reward
print("Reward:", reward)
print()
print("Training Completed")
print("Total Reward:", total_reward)

```

OUTPUT:



```

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Action: Increase Price
Reward: -5

Episode: 4
State: High Demand
Action: Assign Driver
Reward: -5

Episode: 5
State: Medium Demand
Action: Assign Driver
Reward: 8

Episode: 6
State: High Demand
Action: Increase Price
Reward: 10

Episode: 7
State: Low Demand
Action: Increase Price
Reward: -5

Episode: 8
State: High Demand
Action: Increase Price
Reward: 10

Episode: 9
State: High Demand
Action: Assign Driver
Reward: -5

Episode: 10
State: Medium Demand
Action: Assign Driver
Reward: 8

Training Completed
Total Reward: 21
>>>

```